

Improving Genetic Algorithms for Product Storage Allocation

Bedford, Ashley

TSYS School of Computer Science

Columbus State University

Columbus, Georgia

bedford_ashley@columbusstate.edu

Foy, Nia

TSYS School of Computer Science

Columbus State University

Columbus, Georgia

foy_nia@columbusstate.edu

Gaba, Mayank

TSYS School of Computer Science

Columbus State University

Columbus, Georgia

gaba_mayank@columbusstate.edu

Abstract—Improving Genetic Algorithms are important to maximize their full potential in product storage allocation models

Genetic algorithms refer to a process that aims to mimic natural selection. These algorithms draw heavily from the Darwinist technique based on “Survival of the Fittest” to find optimized solutions to quantifiable problems. These algorithms function by modeling the solutions as a population that undergoes a series of changes which produce child solutions that are evaluated. Based on the evaluation, optimal solutions are chosen to be part of the following population and the weaker solutions are eliminated. This cycle is repeated until a stopping criterion is met or the total number of repetitions has been completed.

Each step of this algorithm can be fine-tuned for the specific problem statement. The first step requires the problem to be represented in a quantifiable method that allows for computation. For this research, we will solve the warehouse product storage allocation problem to determine which methodology produces the most optimized model. This refers to how many industries conduct their storage and distribution processes. Most companies today must hire a team of employees to manually assign a container full of product at the port of destination to a warehouse building using tools like Excel to track and manage the effects of their product storage allocation decisions on warehouse space, labor, and product demurrage.

We will aim to improve our Genetic Algorithm in the many ways that we have laid out in our Literature Review, such as implementing a Quantum Strategy for Population Initialization and using One-point Crossover Selection Bias, to see which method is the most effective in providing a tool for companies implementing a Product storage allocation model for their warehouses. The effectiveness of each method will be determined by the population that has the fittest individuals, using our fitness function, and the time it takes to determine the fittest individuals.

Keywords—Genetic Algorithm, One-point Crossover Bias, Population Improvement, Product storage allocation, Warehousing operations

I. INTRODUCTION

Genetic algorithms refer to a process that aims to mimic natural selection. These algorithms draw heavily from the Darwinist technique based on “Survival of the Fittest” to find optimized solutions to quantifiable problems. These algorithms function by modeling the solutions as a population that undergoes a series of changes that produce child solutions that are evaluated. Based on the evaluation, optimal solutions are chosen to be part of the following population and the weaker children are eliminated. This cycle is repeated until a stopping criterion is met or the total number of repetitions has been completed.

Each step of this algorithm can be fine-tuned for the specific problem statement. The first step requires the problem to be represented in a quantifiable method that allows for computation. This involves carefully choosing the data structure to represent the population of solutions. Once this representation is completed and the population has been initialized, how the population will undergo changes needs to be defined. This evolution of the population can be done through various forms of mating operations from the existing population as well as random or defined mutation protocols where the children from the initial population offer different solutions. How these methods are outlined can be vital for optimizing the genetic algorithm. Another major aspect of these algorithms is how to ensure the next population provides solutions that better fit the desired outcome. Although there is a substantial amount of evidence to support that this application is effective, Genetic Algorithms still have opportunities for improvement. The two high-level sections of GAs are Population Initialization and Population Evolution, which include Parent Selection, Crossover, Mutation, Recombination, and Survivor Selection.

For this research, we will solve the warehouse product storage allocation problem to assess which methodology produces the most optimized model. This refers to how many industries conduct their storage and distribution processes. Most companies today manage their warehouses using a software system; however, many of these systems lack the tools needed to aid warehouse departments in optimizing their warehouse storage space. Warehouse and operations teams leave those decisions to their employees working on the floor which may cause bin and overall space underutilization. This research will implement a genetic algorithm to solve this problem and analyze the various improvements suggested by ongoing optimization techniques relevant to genetic algorithms.

Our team will propose the use of Genetic Algorithms for product storage methods which can help companies better conduct their operations. We will aim to improve our Genetic Algorithm in the many ways that we have laid out in our Literature Review, such as implementing a Quantum Strategy for Population Initialization and using One-point Crossover Selection Bias, to see which method is the most effective in providing a tool for companies implementing a product storage allocation model for their warehouses. The results from these will then be compared against each other and all improvement methodologies will be analyzed with respect to the time and space complexity of the algorithm, the accuracy of the result and the simplicity of the algorithms.

II. GENETIC ALGORITHM IMPROVEMENT METHODS

A. Strong Population Initialization

Kim [2] provide a new Quantum Computing approach to improve Genetic Algorithms that is understudied. The authors focus on optimizing a Genetic Algorithm by initializing the population with strong solutions upfront will provide stronger results throughout the Population Evolution phase. Using the Knapsack Problem, [2] present the implementation of their solution. An oracle is created with the Glover Algorithm, which evaluates a set of function inputs by a counting procedure to find a viable solution to our Knapsack problem. All solutions in a starting population are evaluated using a hyperparameter p to determine if enough strong solutions are present. If so, we can continue the problem using a Genetic Algorithm. Otherwise, we will repeat these steps until a strong population exists. A clear advantage to this approach is the runtime being $\theta(\sqrt{N})$ time, which beats classical approaches in quadratic time. Kim [2] ran an experiment to test the effectiveness of this approach against classical GAs. Most of the trials showed a significant increase in the average fitness values of a population, so long as the hyperparameter p value is set appropriately.

B. Strong Parent Selection Bias

Exploring crossover bias, specifically in tree-based Genetic Algorithms should be focused on as it is an opportunity for optimization that is overlooked, as argued by [3]. As a component of Population Evolution, One-point Crossover Selection creates bias towards a set of similar solutions in a Genetic Algorithm. The authors suggest that this bias can be leveraged to produce stronger offspring using tree asymmetry. McPhee [3] explain that there is evidence to support the root parent node having a heavier influence on the strength of an

offspring than a non-root parent node, which can result in the reproduction of stronger offspring. The authors conduct an experiment to see the effectiveness of different crossover bias levels, zero meaning each parent gets a 50/50 split chance of being chosen as the root parent regardless of fitness and one meaning the fittest parent was always chosen as the root parent. Utilizing with crossover bias technique, also known as 100% bias towards the most fit parent to be selected as the root node, resulted in the average fitness values of the population increasing as well as the absolute error over generations decreasing at a faster rate.

This approach does not evaluate function inputs like the solution provided by [2], it evaluates the function itself based on the structure of the evolved trees. As each generation passes, the tree shifts to promote the fittest individual towards the root node. With a different approach comes a separate set of challenges for its implementation. With the increase of crossover bias follows an increase in selection pressure which could limit population diversity over time thus potentially leading to premature convergence as noted by [3]. The effect of this approach carries on to the rest of the Genetic Algorithm sub-methods, like the solution from [2], but could have a larger negative effect on the results. One popular application of [3] approach is through Machine Learning (ML) Model Feature Selection. However, [1] reaffirm that the negative effects of one-point crossover bias extend into this real-world application with results showing an increased bias towards including more features to be used.

III. PROJECT

Our team modelled a real-world storage area in a warehouse using Python. The goal of this program is to maximize the volume of the storage area with a set number of boxes. We separated our storage area into three sub-areas called Zones. In the real-world companies separate their storage areas to optimize their capacity. A Zone class is created with three subclasses known as “Small”, “Medium”, and “Large”. A set number of boxes are created must fit in their assigned Small, Medium, or Large Zone. We created square boxes of the same size per Zone, so that we can account for its orientation when filling the Zone’s available volume. Using a Genetic Algorithm that completes 5,000 iterations, we were able to find a group of boxes that fills the volume of each Zone to the highest level possible. Additionally, we explored the Genetic Algorithm Improvement Methods proposed by other authors to determine which method(s) produced the strongest results.

We created a Standard Genetic Algorithm (GA) which includes the following components:

- **Population Initialization:** The program starts by generating randomized bits to create one population per Zone
- **Parent Selection:** Four random chromosomes are chosen to potentially create new chromosomes known as children. A reproduction rate (REPRO_RATE) is set which lets the program create new children only if we reach or exceed the probability of this rate using the random.random function. If the reproduction rate is met or exceeded, we will advance to the next step below, otherwise, our parents will remain in our

population for the next Generation Cycle, and we will skip the Crossover and Mutation parts.

- **Fitness Evaluation:** Each chromosome in our population is evaluated to see what percentage of our available volume is filled. This percentage is called a fitness value and is associated with each chromosome. In every generation, the largest fitness value is stored and plotted over each Generation Cycle.
- **Crossover:** If we advanced to this step, we will evaluate our crossover rate (CROSS_RATE). If the random.random function meets or exceeds this rate, then the new children are created using a One-point Crossover method, else we move on to the next step. This method uses a random index and splits two parents and swaps one side of parent one with the opposite side of parent two to create a child. Six children are created from four parents.
- **Mutation:** If the random.random meets or exceeds the mutation rate (MUTATE_RATE), then we will conduct a one-bit flip method. This method iterates through each gene of a child chromosome and if random.random meets or exceeds the mutation rate, then the gene “flips”, meaning we change a 1 to 0 and vice-versa.
- **Survivor Selection:** If 6 children are created, and 4 randomly chosen solutions from the previous generation survive. Parents are eligible to be chosen again to survive. Else the 4 parents continue on to the next generation cycle and 6 randomly chosen solutions from the previous cycle continue on to the next generation cycle.
- **Stopping Criteria:** Then this cycle continues from Parent Selection to Survivor Selection until the number of iterations have completed.

IV. RESULTS

After running this Standard GA, we received the result below.

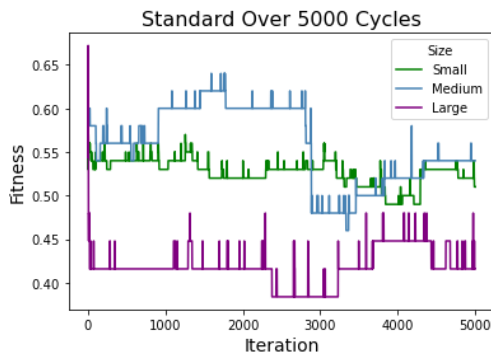


Fig. 1. Standard GA Fitness Values over 5,000 Cycles.

On average, our aggregate results could not break through 50% volume utilization of our Zones. These results are not the best, since they leave a significant amount of space unused, and each Zone’s highest fitness value seems to stay around the same

range of values over time with a few critical variations. We continued testing the proposed GA improvements.

A. Strong Population Initialization

We created a separate model that starts off with a strong population like the approach that [2] provided. Using the Standard GA we created, we solely altered the Population Initialization portion by only accepting chromosomes that produced a fitness value above 85%.

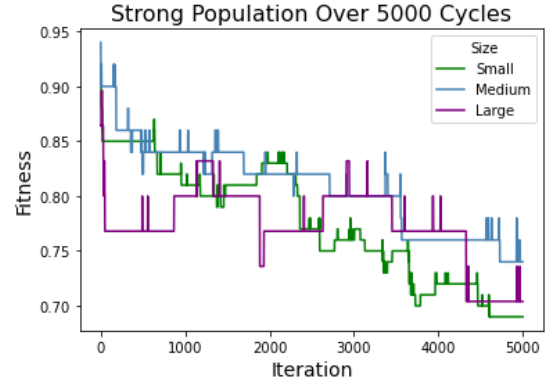


Fig. 2. Strong Population GA Fitness Values over 5,000 Cycles.

As a result, we found that our Genetic Algorithm’s fitness function decreased over time. This result may have been due to the Crossover and Mutation portions of our program since children are created from modified versions of their strong initial population. This solidifies our theory that strong initial populations do not guarantee strong populations over time.

B. Strong Parent Selection Bias

To implement the second technique as presented by [3], our team modified the Parent Selection portion of our program. The results are shown below.

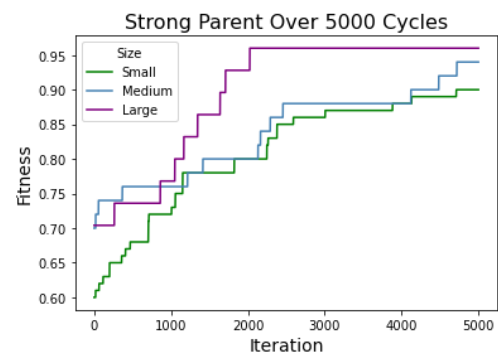


Fig. 3. Strong Parent Fitness Values over 5,000 Cycles.

Instead of randomly selecting parents in our population, we chose parents based on their fitness values. Then, these strong parents would go through a one-point crossover and mutation method to generate children. The results from this method proved to provide consistent growth. Each of the Zones reached

a high fitness value of 90% or over and all three lines show an exponential increase in our fitness value over each cycle.

C. Strong Population Initialization & Parent Selection Bias

After reviewing the results from techniques, A and B, our team wanted to test how both improvements would affect our results. We decided to revise our Standard GA to set a strong initial population and have a strong parent selection method.

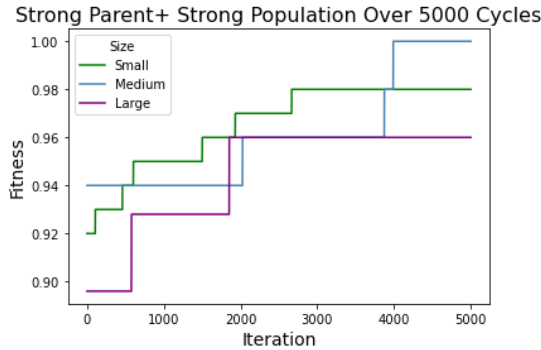


Fig. 4. Strong Population with Strong Parent GA Fitness Values over 5,000 Cycles.

Our results show a significant improvement in the highest fitness functions over time. Also, this methodology proved to reach the highest achievable result in the smallest number of iterations.

V. CONCLUSION

The standard method served as a benchmark for all improvements to be compared to. After 5000 cycles, the standard method provided an average fitness value of 55%. Strong parent selection proved key in maximizing the growth of this algorithm's fitness values. When utilizing this approach, each iteration provided consistent growth in the amount of volume utilized and produced children solutions that proved stronger at each iteration. By using this approach, after 5000 cycles, the algorithm produced a population with a max fitness value of over 95%.

Using a strong initial population also provided results that proved better than the standard. Although this method led to an initial population with over 90% fitness value, over 5000 cycles, this value decayed to an average of 75%. This decay was mainly because the standard algorithm chose parents at random, which led to the population consistently containing solutions that were not optimal. With absolutely no doubt, the combination of consistently using strong parents with an initial strong population provided the fastest growth to the maximum potential. This is due to primarily how we structured our algorithm to favor utilizing a solution that maximizes the volume used. Another factor is how we structured our inputs to reflect the total maximum capacity for the environment defined. This method provided an initial population with over 90%

fitness value and in under 3000 cycles achieved a fitness value of over 96% and by 5000 cycles, achieved an average value of 98%.

Genetic algorithms function by modeling the solutions as a population that undergoes a series of changes that produce child solutions that are evaluated. Based on the evaluation, optimal solutions are chosen to be a part of the following population and the weaker solutions are eliminated. This cycle is repeated until a stopping criterion is met or the total number of repetitions has been completed. Each step of genetic algorithms can be fine-tuned for the specific problem statement. This research was focused on creating a generic genetic algorithm that attempts to automate the storage processes conducted by many industries. This algorithm was equipped with various optimized methodologies outlined above. The results from these improvement methods showed that using a strong initial population with a key focus on parent selection led to the fastest growth to the achievable potential.

REFERENCES

- [1] Lauro C.M. de Paula, Anderson S. Soares, Telma Woerle de Lima, and Clarimar J. Coelho. 2016. Feature Selection using Genetic Algorithm: An Analysis of the Bias-Property for One-Point Crossover. In Proceedings of the 2016 on Genetic and Evolutionary Computation Conference Companion (GECCO '16 Companion). Association for Computing Machinery, New York, NY, USA, 1461–1462. <https://doi.org/10.1145/2908961.2931636>
- [2] Jun Suk Kim and Chang Wook Ahn. 2022. Quantum strategy of population initialization in genetic algorithm. In Proceedings of the Genetic and Evolutionary Computation Conference Companion (GECCO '22). Association for Computing Machinery, New York, NY, USA, 439–442. <https://doi.org/10.1145/3520304.3529010>
- [3] Nicholas Freitag McPhee, M. Kirbie Dramdahl, and David Donatucci. 2015. Impact of Crossover Bias in Genetic Programming. In Proceedings of the 2015 Annual Conference on Genetic and Evolutionary Computation (GECCO '15). Association for Computing Machinery, New York, NY, USA, 1079–1086. <https://doi.org/10.1145/2739480.2754778>
- [4] V. E. Torchinskii, O. S. Logunova, N. S. Sibileva, and P. Yu. Romanov. 2018. Genetic algorithm modification: addition of the population improvement stage. In Proceedings of the 2018 International Conference on Information Science and System (ICISS '18). Association for Computing Machinery, New York, NY, USA, 286–290. <https://doi.org/10.1145/3209914.3209928>
- [5] Cetin, U., & Gundogmus, Y. E. (2020). Feature Selection with Evolving, Fast and Slow Using Two Parallel Genetic Algorithms. arXiv. <https://doi.org/10.1109/UBMK.2019.8907165>
- [6] Lu, T., Cen, M., Huo, S., & Luo, L. (2022). Principal component analysis-improved fuzzy genetic algorithm. Proceedings of the 2022 2nd International Conference on Control and Intelligent Robotics. <https://doi.org/10.1145/3548608.3559203>
- [7] Philemotte, C., & Bersini, H. (2007). A gestalt genetic algorithm. Proceedings of the 9th Annual Conference on Genetic and Evolutionary Computation - GECCO '07. <https://doi.org/10.1145/1276958.1277208>
- [8] Ayachi, I., Kammarti, R., Ksouri, M., & Borne, P. (2010). A Genetic algorithm to solve the product storage space allocation problem. 2010 International Conference on Computational Intelligence and Vehicular System (CIVS), 1–4.
- [9] Bazzazi, M., Safaei, N., & Javadian, N. (2009). A genetic algorithm to solve the storage space allocation problem in a product terminal. Computers & Industrial Engineering, 56(1), 44–52. <https://doi.org/10.1016/j.cie.2008.03.012>

[10] Yan Xu, Chengxuan Cao, Bin Jia, Guangzhi Zang, "Model and Algorithm for Product storage allocation Problem with Random Freight Demands in Synchromodal Transportation", Mathematical Problems in Engineering,

vol. 2015, Article ID 986152, 13 pages, 2015.
<https://doi.org/10.1155/2015/986152>